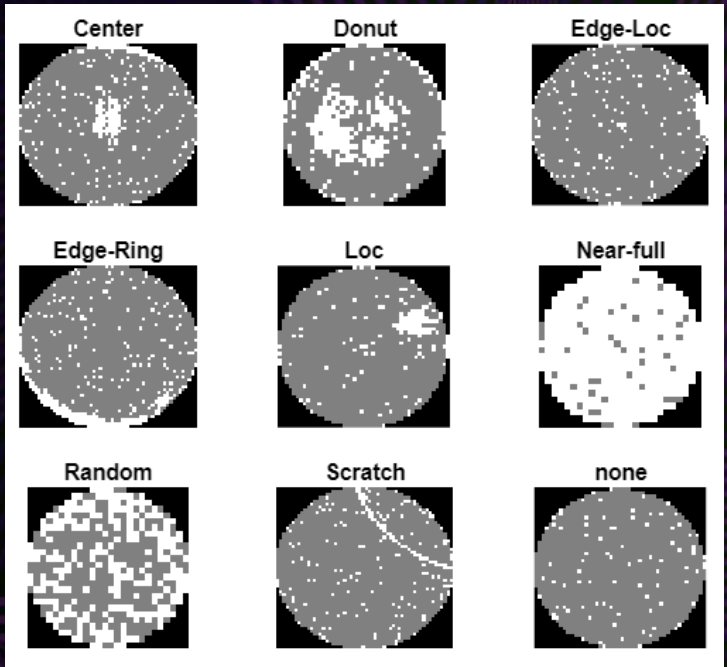
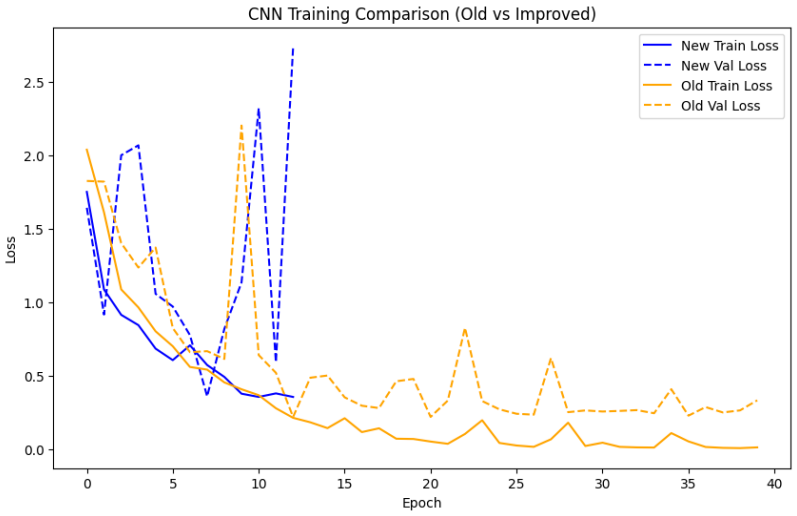


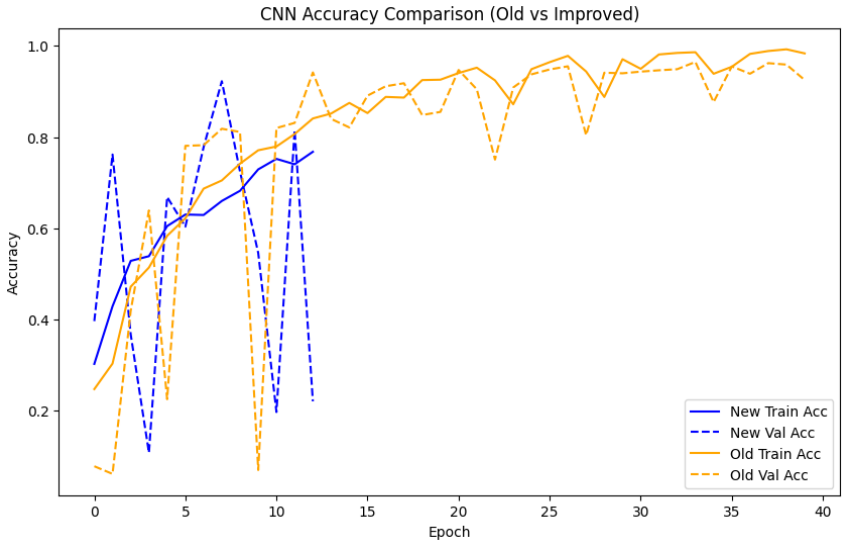

# Wafer Defect Classification Final Milestone

BY: NAJANI JOHNSON AND MEGHAN HERBERT

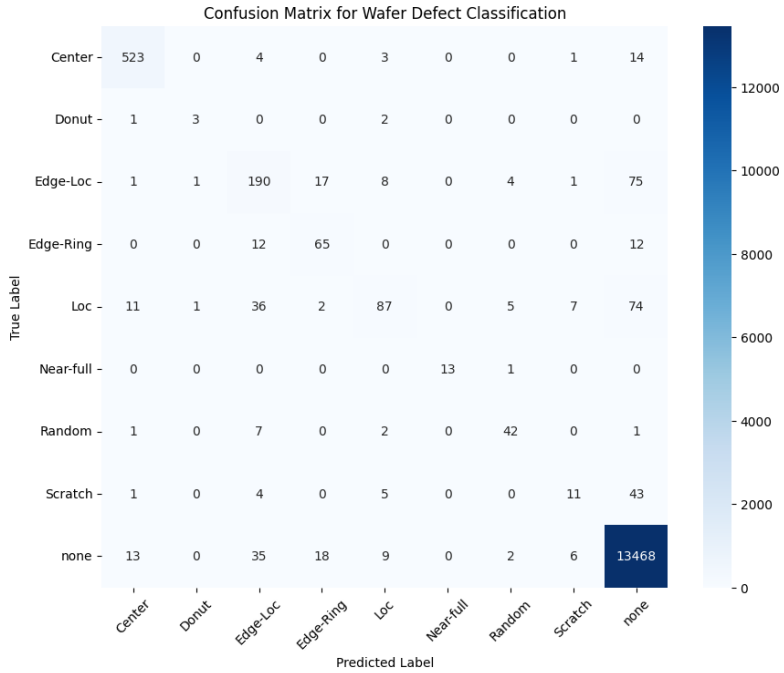
# Previous Milestone Recap

## CNN Training & Accuracy Comparison Graphs



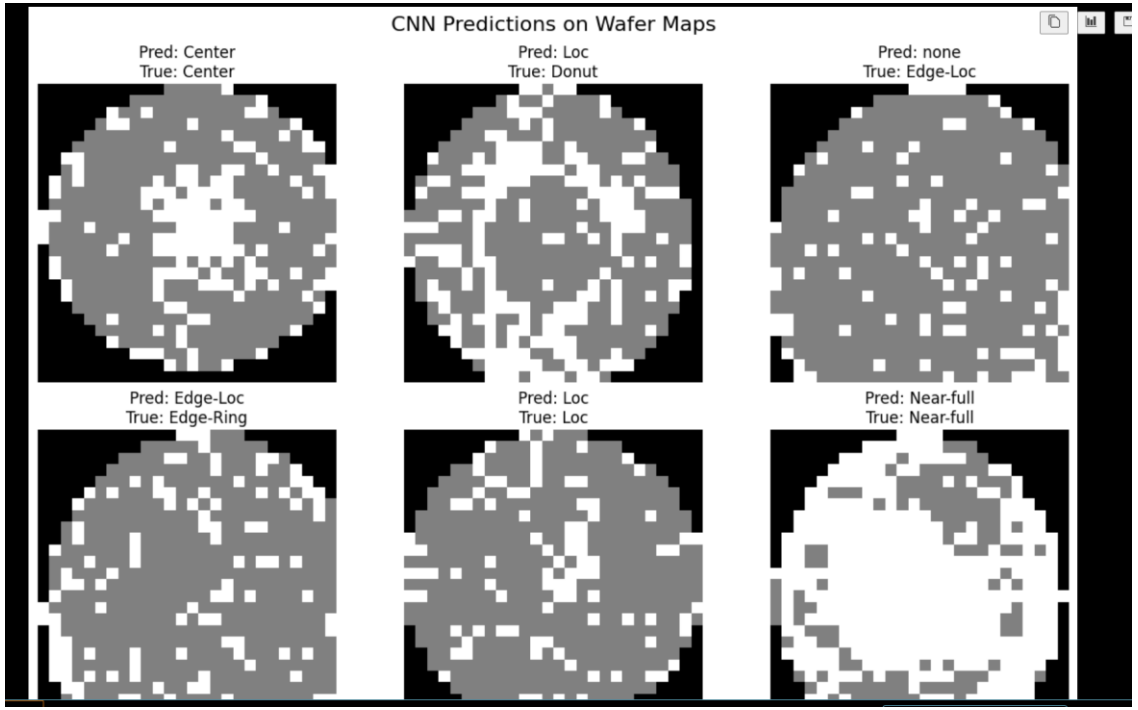




## Confusion Matrix



# Previous Milestone Key Limitations

- Inconsistent validation setup
- CNN architecture limitations
- Class imbalance effects




*Model went 3/6 on unseen data. We believe our accuracy isn't fitting of the full context because our old model does great on none wafers and the majority of the dataset is none wafers*

```python
from sklearn.model_selection import train_test_split

X = X.astype("float32") / 2.0
X = X[..., np.newaxis]

X_train_full, X_test, y_train_full, y_test = train_test_split(
    X,
    y_encoded,
    test_size=0.2,
    random_state=42,
    stratify=y_encoded
)

X_train, X_val, y_train, y_val = train_test_split(
    X_train_full,
    y_train_full,
    test_size=0.2,
    random_state=42,
    stratify=y_train_full
)
```

[133] Python

```python
X_train_flat = X_train_full.reshape(len(X_train_full), -1)
X_test_flat = X_test.reshape(len(X_test), -1)
```

[134] Python

## Baseline Model: Random Forest Classifier

Before applying deep learning, we first train a traditional machine learning model as a baseline.

A **Random Forest classifier** is used because it performs well on many classification tasks and requires minimal feature engineering. Since classical models expect one-dimensional input vectors, each wafer map is flattened into a single feature vector.

The performance of this baseline model provides a reference point for evaluating the effectiveness of the convolutional neural network introduced later.

```python
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=100, n_jobs=-1)
rf.fit(X_train_flat, y_train_full)
pred = rf.predict(X_test_flat)
```

[135] Python

# Fixing Random Forest Input Formatting

- FLATTENED X_TRAIN_FULL FOR RANDOM FOREST
- USED MATCHING LABELS: Y_TRAIN_FULL
- AVOIDED USING SMALLER CNN SPLIT LABELS
- KEPT THE BASELINE VALID

# Fixing Inconsistent Validation Setup

## BEFORE

```
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.model_selection import train_test_split

X = X.astype("float32") / 2.0
X = X[..., np.newaxis]

X_train_full, X_test, y_train_full, y_test = train_test_split(
    X,
    y_encoded,
    test_size=0.2,
    random_state=42,
    stratify=y_encoded
)

early_stop = EarlyStopping(
    patience=5,
    restore_best_weights=True,
    monitor='val_loss',
    verbose=1
)

history = model.fit(
    X_train,
    y_train,
    epochs=40,
    batch_size=64,
    validation_data=(X_val, y_val),
    callbacks=[early_stop],
    class_weight=class_weights
)

print(history.history['val_loss'])
```

## AFTER

```
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

early_stop = EarlyStopping(
    patience=5,
    restore_best_weights=True,
    monitor="val_loss",
    verbose=1
)

reduce_lr = ReduceLROnPlateau(
    monitor="val_loss",
    factor=0.5,
    patience=3,
    min_lr=1e-6,
    verbose=1
)

history = model.fit(
    X_train,
    y_train,
    epochs=40,
    batch_size=64,
    validation_data=(X_val, y_val),
    callbacks=[early_stop, reduce_lr],
    class_weight=class_weights
)

print(history.history["val_loss"])
```

- Added stratified train/test splitting
- Created one consistent validation set
- Stopped re-splitting data during model training
- Added ReduceLROnPlateau with early stopping
- Made old vs improved CNN comparison fairer

# Addressing Class Imbalance: Oversampling vs Weights (part 1)

## Adding Weights Snippet:

```python
# compute class weights
weights = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(y_train),
    y=y_train
)

# convert to dictionary format
class_weights = dict(enumerate(weights))
```

## Oversampling Snippet:

```python
X_balanced = []
y_balanced = []

classes = np.unique(y_train)

# choose target count per class
target_count = 5000

for c in classes:
    X_c = X_train[y_train == c]
    y_c = y_train[y_train == c]

    if len(y_c) < target_count:
        X_resampled, y_resampled = resample(
            X_c,
            y_c,
            replace=True,
            n_samples=target_count,
            random_state=42
        )
    else:
        X_resampled, y_resampled = resample(
            X_c,
            y_c,
            replace=False,
            n_samples=target_count,
            random_state=42
        )

    X_balanced.append(X_resampled)
    y_balanced.append(y_resampled)

X_train_balanced = np.concatenate(X_balanced)
y_train_balanced = np.concatenate(y_balanced)

# shuffle balanced training data
shuffle_idx = np.random.RandomState(42).permutation(len(y_train_balanced))
X_train_balanced = X_train_balanced[shuffle_idx]
y_train_balanced = y_train_balanced[shuffle_idx]
```

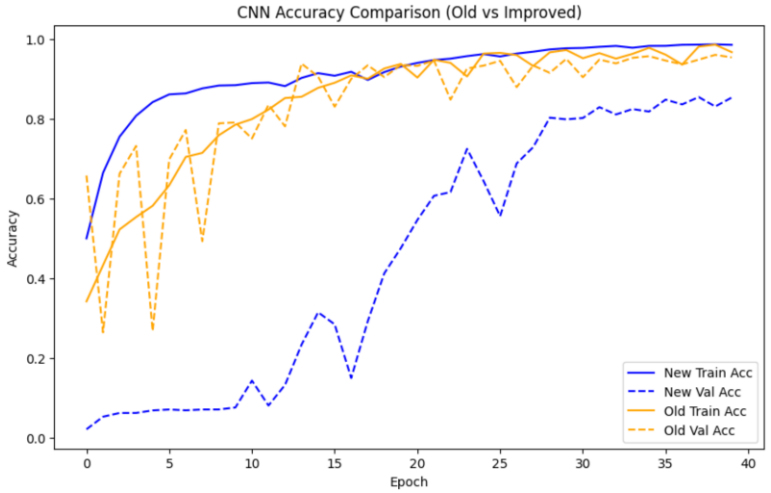
## Insights:

- **Weights:** adjust the loss function during training
- **Weights:** rare classes count more when errors happen
- **Oversampling:** increases rare-class sample counts
- **Oversampling:** repeats existing minority-class wafers
- **Both:** aim to reduce majority-class bias

# Addressing Class Imbalance: Oversampling vs Weights (part 2)

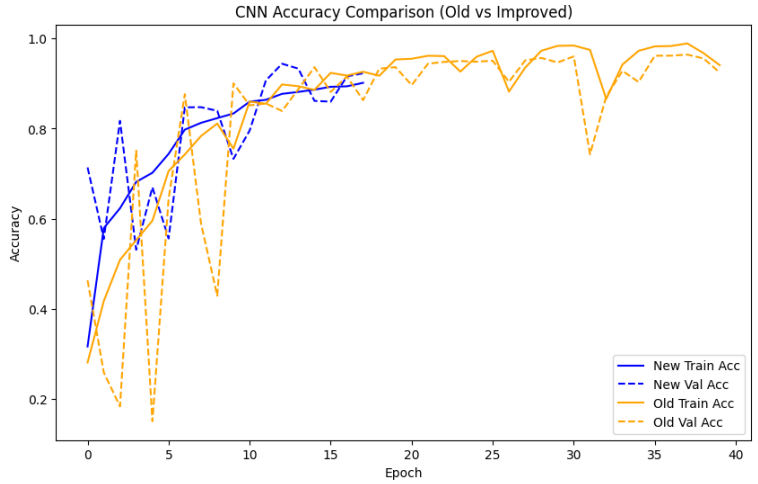
## Data from Oversampling

| | Model | Accuracy | F1 Score |
|---|---|---|---|
| 0 | Random Forest | 0.9583 | 0.9476 |
| 1 | CNN (Old) | 0.9523 | 0.9567 |
| 2 | CNN (Improved) | 0.8527 | 0.8892 |



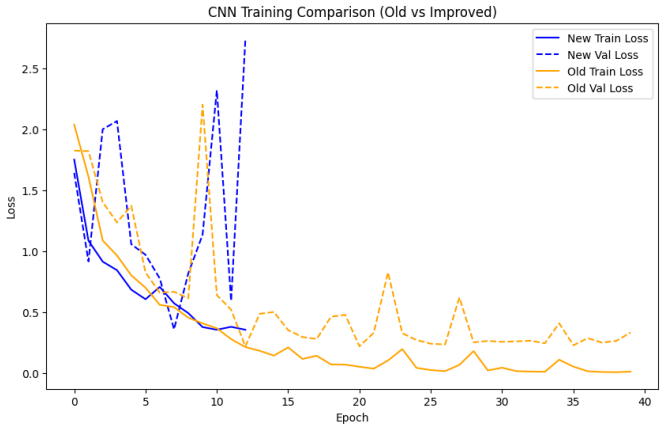

## Data from adding Weights

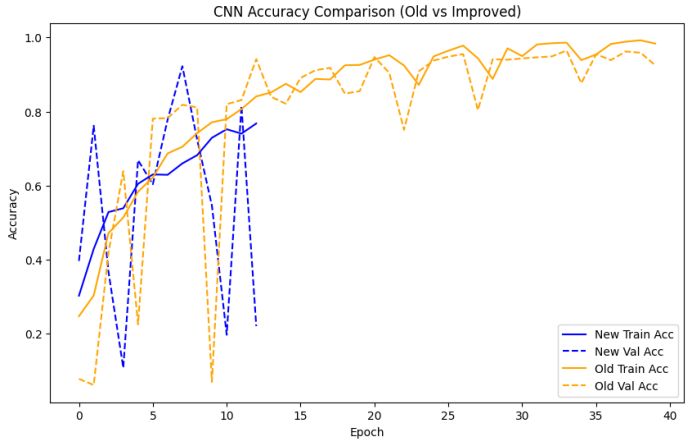
| | Model | Accuracy | F1 Score |
|---|---|---|---|
| 0 | Random Forest | 0.9590 | 0.9487 |
| 1 | CNN (Old) | 0.9272 | 0.9391 |
| 2 | CNN (Improved) | 0.9388 | 0.9476 |



# Results of All Fixes

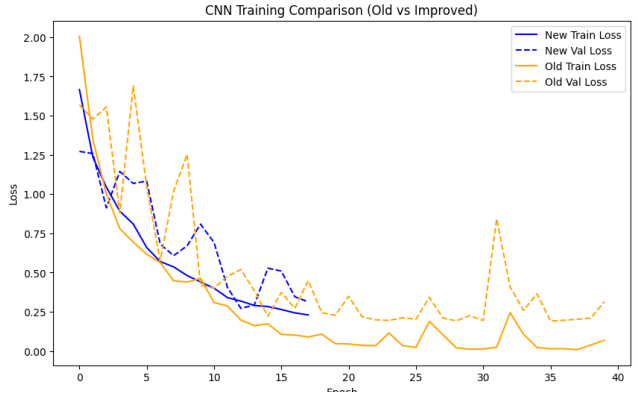
## Before

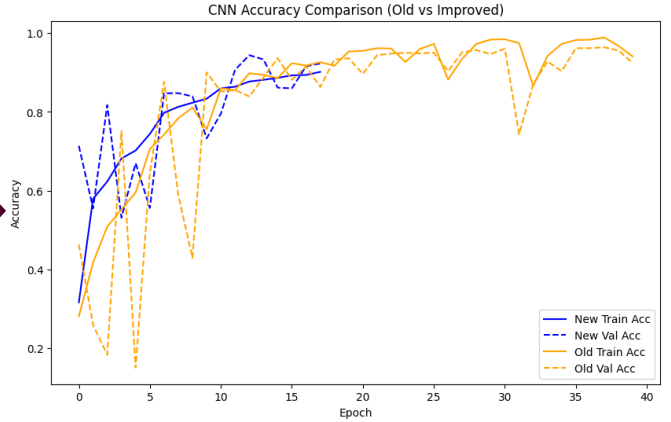


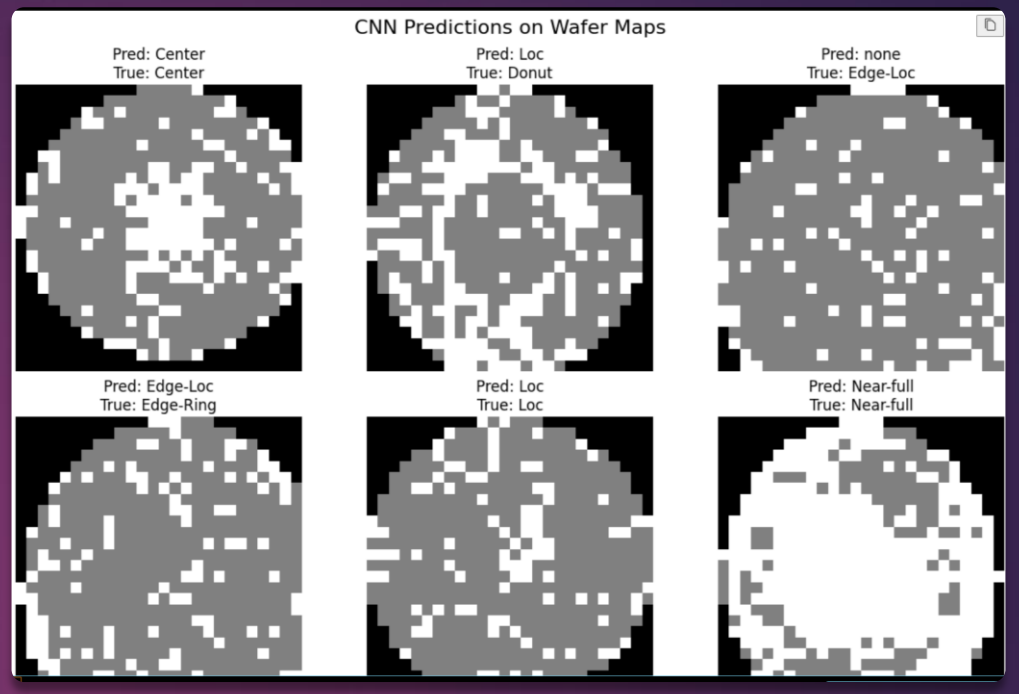

| | Model | Accuracy | F1 Score |
|---|---|---|---|
| 0 | Random Forest | 0.9592 | 0.9479 |
| 1 | CNN (Old) | 0.9216 | 0.9346 |
| 2 | CNN (Improved) | 0.9280 | 0.9334 |

## After




| | Model | Accuracy | F1 Score |
|---|---|---|---|
| 0 | Random Forest | 0.9590 | 0.9487 |
| 1 | CNN (Old) | 0.9272 | 0.9391 |
| 2 | CNN (Improved) | 0.9388 | 0.9476 |


CNN Predictions on Wafer Maps
Pred: Center
True: Center
Pred: Loc
True: Donut
Pred: none
True: Edge-Loc
Pred: Edge-Loc
True: Edge-Ring
Pred: Loc
True: Loc
Pred: Near-full
True: Near-full

# DEMO

# Key Takeaways and Next Steps

**Key Takeaways**

- Cleaner splits made model comparisons more reliable
- Random Forest remained the strongest baseline
- Improved CNN performed better with class weights than oversampling
- Class imbalance still affects rare defect classes

**Next Steps**

- Tune CNN architecture and learning rate further
- Track per-class recall and F1 more closely
- Test stronger imbalance methods beyond simple oversampling
- Add more validation with unseen wafer-map examples